Python Zero to Projects

Chapter 4: Strings

Study Notes

In Chapter 2 you learned how variables store data such as numbers and text, and in Chapter 3 you learned how operators like **+**, *****, **==**, and **!=** work on numbers. It turns out many of those same operators also work on text. This chapter takes a deep dive into the **str** (string) data type: how to create strings, read individual characters out of them, combine them, transform them with built-in methods, and format them neatly for output.

What you will learn in this chapter:

- Creating strings and measuring their length with len()
- Indexing and slicing to pull out characters or substrings
- Concatenation, repetition, and the in / not in membership operators
- Why strings are immutable, and what that means in practice
- Six essential string methods: upper, lower, strip, split, replace, find
- Three ways to format strings: % operator, .format(), and f-strings
- Multiline strings and escape characters
- Common beginner mistakes -- and how to avoid them

1. Creating Strings and Finding Their Length

A **string** is a sequence of characters used to represent text. Just like you assigned numbers to variables in Chapter 2 (for example `age = 16`), you can assign text to a variable by wrapping it in quotes.

Three ways to write a string literal:

- **Single quotes:** 'Hello' -- good for short strings
- **Double quotes:** "Hello" -- behaves identically to single quotes; useful when the text itself contains a single quote, e.g. "It's sunny"
- **Triple quotes:** '''Hello''' or """Hello""" -- allow the string to span multiple lines (covered in detail in Section 7)

Example

```
first_name = "Maya"
last_name = 'Khan'
greeting = "It's a great day to learn Python!"

print(first_name)
print(last_name)
print(greeting)
```

Output

```
Maya
Khan
It's a great day to learn Python!
```

Notice that `first_name` and `last_name` are variables, exactly like the ones you created in Chapter 2 -- the only difference is that the value stored in them is text (a `str`) instead of a number.

The len() function

The built-in `len()` function returns the number of characters in a string, including spaces and punctuation.

Example

```
language = "Python"
sentence = "I love coding"

print(len(language))
print(len(sentence))
```

Output

```
6
13
```

Count the characters in "I love coding" yourself: the spaces between words are counted too, which is why the length is 13 and not 11.

2. Indexing and Slicing

Because a string is a sequence of characters, every character has a numbered position called an **index**. Python supports two kinds of indices: **positive** indices that count from the left starting at 0, and **negative** indices that count from the right starting at -1.

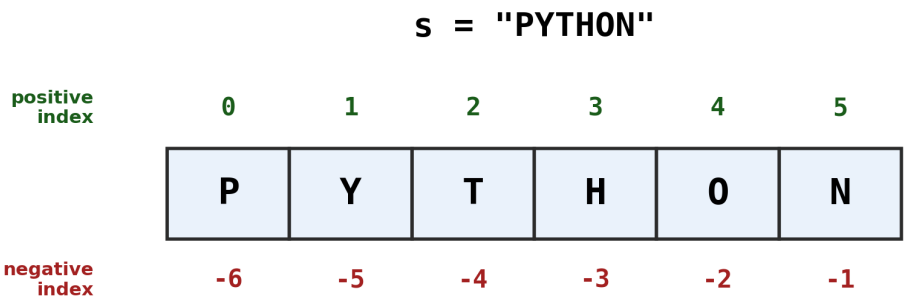

Figure 1: Positive indices count from the left starting at 0; negative indices count from the right starting at -1.

2.1 Indexing -- accessing a single character

Use square brackets `[]` with an index to pull out one character.

```
s = "PYTHON"

print(s[0])      # first character
print(s[2])      # third character
print(s[-1])     # last character
print(s[-6])     # first character, counted from the right
```

Output

```
P
T
N
P
```

Trying to access an index that does not exist (for example `s[6]` on a 6-character string, since the last valid index is 5) raises an `IndexError`. More on this common mistake in Section 8.

2.2 Slicing -- accessing a range of characters

Slicing uses the form `s[start:stop:step]` to extract a substring. **start** is the first index included, **stop** is the first index NOT included (the slice stops right before it), and **step** controls how many characters to skip between selections (it defaults to 1 and can be omitted).

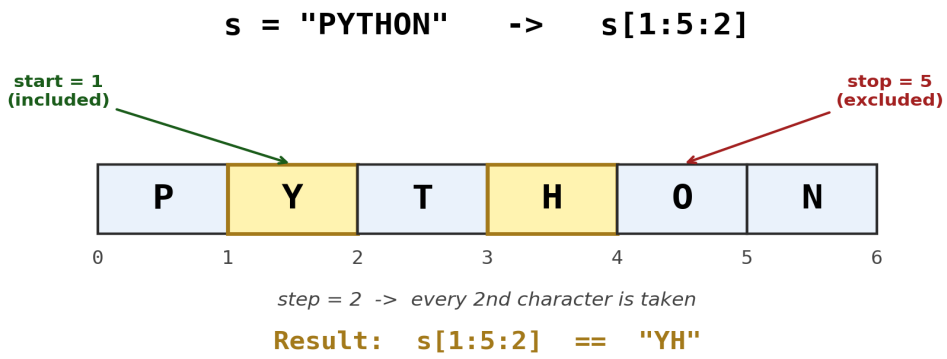

Figure 2: s[1:5:2] starts at index 1 (included), stops before index 5 (excluded), and takes every 2nd character in between.

Example

```
s = "PYTHON"

print(s[1:4])     # characters at index 1, 2, 3
print(s[:3])       # from the start up to (not including) index 3
print(s[3:])       # from index 3 to the end
print(s[:])         # the whole string (a copy)
print(s[::-1])     # the whole string, reversed (step = -1)
print(s[1:5:2])    # index 1 and index 3 only
```

Output

```
YTH
PYT
HON
PYTHON
NOHTYP
YH
```

Tip

When start is left out, Python assumes the very beginning of the string. When stop is left out, Python assumes the very end. A step of -1 walks backwards through the string, which is a quick way to reverse it.

3. Concatenation, Repetition, and Membership

In Chapter 3 you used + and * on numbers for addition and multiplication. On strings, these same operators do something different but equally useful.

3.1 Concatenation with +

The + operator joins (concatenates) two strings into one.

```
first_name = "Maya"
last_name = "Khan"

full_name = first_name + " " + last_name
print(full_name)
```

Output

```
Maya Khan
```

3.2 Repetition with *

The * operator repeats a string a given number of times.

```
border = "-" * 20
print(border)
print("ha" * 3)
```

Output

```
--------------------
hahaha
```

3.3 Membership: in and not in

The `in` and `not in` operators (which behave like the comparison operators == and `!=` from Chapter 3 in that they also return a boolean) check whether one string appears inside another.

```
sentence = "I love coding in Python"

print("Python" in sentence)
print("Java" in sentence)
print("Java" not in sentence)
print(sentence == "I love coding in Python")
print(sentence != "I love coding in Java")
```

Output

```
True
False
True
True
True
```

Just like numeric comparisons, `in`, `not in`, ==, and `!=` always evaluate to a `bool` (True or False) -- the same data type you first met in Chapter 2.

4. String Immutability

Strings in Python are **immutable**, which means that once a string is created, its characters cannot be changed in place. Any operation that looks like it is "modifying" a string is actually building a brand new string in memory.

This will raise an error:

```
s = "Python"
s[0] = "J"          # attempting to change the first character
```

Output

```
Traceback (most recent call last):
  File "main.py", line 2, in <module>
    s[0] = "J"
TypeError: 'str' object does not support item assignment
```

To get a "changed" version of a string, you must create a new string and (usually) reassign it to the same variable name:

```
s = "Python"
s = "J" + s[1:]      # build a brand new string instead
print(s)
```

Output

```
Jython
```

Why does this matter?

Many string methods you are about to meet in Section 5 (such as upper() or replace()) do NOT change the original string -- they return a new one. If you forget to store that returned value, your original variable stays exactly the same.

5. Useful String Methods

A **method** is a function that belongs to a value and is called using dot notation: `value.method()`. Strings come with many built-in methods. Here are six of the most useful ones for a beginner.

5.1 upper() and lower()

Return a new string with all letters converted to uppercase or lowercase.

```
name = "Maya"

print(name.upper())
print(name.lower())
```

Output

```
MAYA
maya
```

5.2 strip()

Returns a new string with leading and trailing whitespace removed (very useful for cleaning up text entered with `input()`, which you met in Chapter 2).

```
raw = "   hello world   "
print("[" + raw.strip() + "]")
```

Output

```
[hello world]
```

5.3 split()

Breaks a string into pieces wherever a separator occurs (a space by default) and returns those pieces as a **list**. Lists are the subject of the very next chapter -- split() is one of the most common ways a list gets created from a string.

```
sentence = "I love coding in Python"
words = sentence.split()
print(words)
print(type(words))

csv_line = "Maya,16,Jaipur"
fields = csv_line.split(",")
print(fields)
```

Output

```
['I', 'love', 'coding', 'in', 'Python']
<class 'list'>
['Maya', '16', 'Jaipur']
```

5.4 replace()

Returns a new string with every occurrence of one substring swapped for another.

```
sentence = "I love coding in Java"
new_sentence = sentence.replace("Java", "Python")
print(new_sentence)
```

Output

```
I love coding in Python
```

5.5 find()

Returns the index of the first occurrence of a substring, or `-1` if the substring is not found (find() never raises an error -- see Section 8 for why this matters).

```
sentence = "I love coding in Python"

print(sentence.find("coding"))
print(sentence.find("Java"))
```

Output

```
7
-1
```

Remember

None of these methods change the original string -- they all return a brand new one. This is a direct consequence of the immutability you learned about in Section 4, so you usually need to store the result in a variable, e.g. name = name.upper().

6. String Formatting

Often you need to build a string that mixes fixed text with variable values, such as printing a name and an age together. Python offers three ways to do this. All three examples below produce the exact same output.

6.1 The % operator (oldest style)

```
name = "Maya"
age = 16

message = "Hello, %s! You are %d years old." % (name, age)
print(message)
```

Output

```
Hello, Maya! You are 16 years old.
```

6.2 The .format() method

```
name = "Maya"
age = 16

message = "Hello, {}! You are {} years old.".format(name, age)
print(message)
```

Output

```
Hello, Maya! You are 16 years old.
```

6.3 f-strings (modern style)

An f-string is written by placing the letter `f` directly before the opening quote. Any variable name placed inside curly braces `{}` is automatically replaced with its value.

```
name = "Maya"
age = 16

message = f"Hello, {name}! You are {age} years old."
print(message)
```

Output

```
Hello, Maya! You are 16 years old.
```

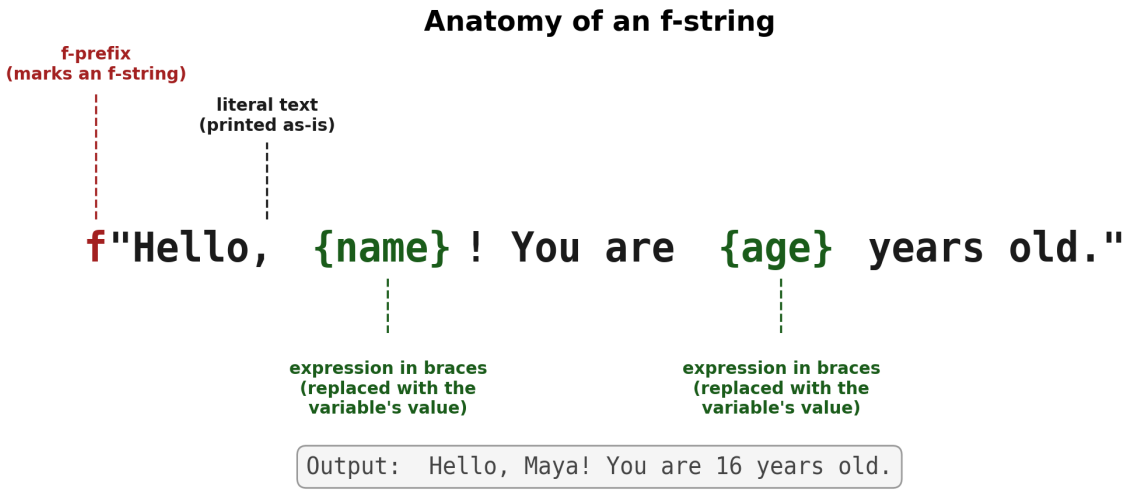

Figure 3: An f-string mixes literal text with expressions inside curly braces; Python evaluates each expression and inserts its value.

> **Best practice**
> f-strings are the modern, preferred way to format strings in Python. They are easier to read, less error-prone (no need to remember the right %-code for each data type), and can even contain small expressions directly, e.g. f"Next year you will be {age + 1}." Prefer f-strings in your own code from this point onward.

7. Multiline Strings and Escape Characters

7.1 Multiline (triple-quoted) strings

Triple quotes (`'''` or `"""`) let a string span several lines exactly as you typed it, including the line breaks.

```
poem = """Roses are red,
Violets are blue,
Python is fun,
And so are you."""

print(poem)
```

Output

```
Roses are red,
Violets are blue,
Python is fun,
And so are you.
```

7.2 Escape characters

An escape character starts with a backslash \ and tells Python to treat the next character specially instead of printing it literally. This is how you insert things like new lines or quote marks inside a normal (single-line) string.

Escape	Meaning	Example code	Effect on output
\n	newline	print("A\nB")	A and B print on separate lines
\t	tab	print("A\tB")	a tab gap is inserted between A and B
\\	backslash	print("C:\\Users")	prints a single backslash: C:\Users
\'	single quote	print('It\'s ok')	lets ' appear inside a '...' string
\"	double quote	print("She said \"hi\"")	lets " appear inside a "..." string

A combined example:

```
print("Name:\tMaya")
print("City:\tJaipur")
print("Line1\nLine2")
print("He said, \"Python is great!\"")
print("Path: C:\\Users\\Maya")
```

Output

```
Name:   Maya
City:   Jaipur
Line1
Line2
He said, "Python is great!"
Path: C:\Users\Maya
```

8. Common Beginner Mistakes

Mistake 1: Trying to modify a string in place

Strings are immutable (Section 4). Writing `s[0] = "J"` raises a `TypeError`. Build a new string instead, e.g. `s = "J" + s[1:]`.

Mistake 2: Off-by-one indexing errors

Indexing starts at 0, not 1. For a 6-character string, the valid positive indices are 0 through 5 -- not 1 through 6. Beginners often expect `s[1]` to be the first character; it is actually the second.

Mistake 3: IndexError on out-of-range indices

Accessing an index beyond the last character (for example `s[6]` on a 6-character string) raises `IndexError: string index out of range`. Slicing, by contrast, never raises this error -- `s[0:100]` simply returns whatever characters exist.

Mistake 4: Forgetting that find() returns -1 instead of raising an error

Unlike indexing, `find()` does not crash when the substring is missing -- it quietly returns `-1`. Beginners sometimes forget to check for this and treat -1 as if it were a valid index.

Mistake 5: Mixing up split() and join()

`split()` turns one string into a list of pieces (string to list). `join()` does the opposite: it glues a list of strings back together into one string, and is called ON the separator, e.g. `" ".join(words)`, not on the list. join() will be covered in more detail once lists are introduced in Chapter 5.

Chapter 4 Recap

- Strings can be written with single, double, or triple quotes; len(s) gives the character count.
- s[i] indexes a single character (positive or negative); s[start:stop:step] slices out a substring.
- + concatenates strings, * repeats them, and in / not in test for a substring -- the same comparison operators (==, !=) from Chapter 3 also work on strings.
- Strings are immutable: methods and slicing always return a NEW string rather than changing the original.
- upper(), lower(), strip(), split(), replace(), and find() are six essential string methods; split() returns a list.
- Strings can be formatted with the % operator, .format(), or f-strings -- f-strings are the modern, preferred choice.
- Triple quotes create multiline strings; escape characters like \n, \t, \\, \', and \" insert special characters into single-line strings.

Looking ahead to Chapter 5: Lists

Chapter 5 introduces lists, Python's go-to container for storing many values together. You will reuse indexing, slicing, and len() in this new context -- they work on lists in almost exactly the same way they work on strings. In fact, you have already created your first lists in this chapter: every call to split() returns one!